Probabilistic Machine Learning

Lecture 04: Discriminative Models: Linear and Logistic Regression

Prof. Dr. Michael Granitzer



Where We Left Off

Lecture 03 built a classifier from probability and took two shortcuts on the way:

Shortcut	Instead of the ideal	What is dropped
Keep the peak	the most probable parameter (MLE, MAP)	the spread: how sure we are
Assume it apart	factorise the joint — naive Bayes	the dependences between attributes

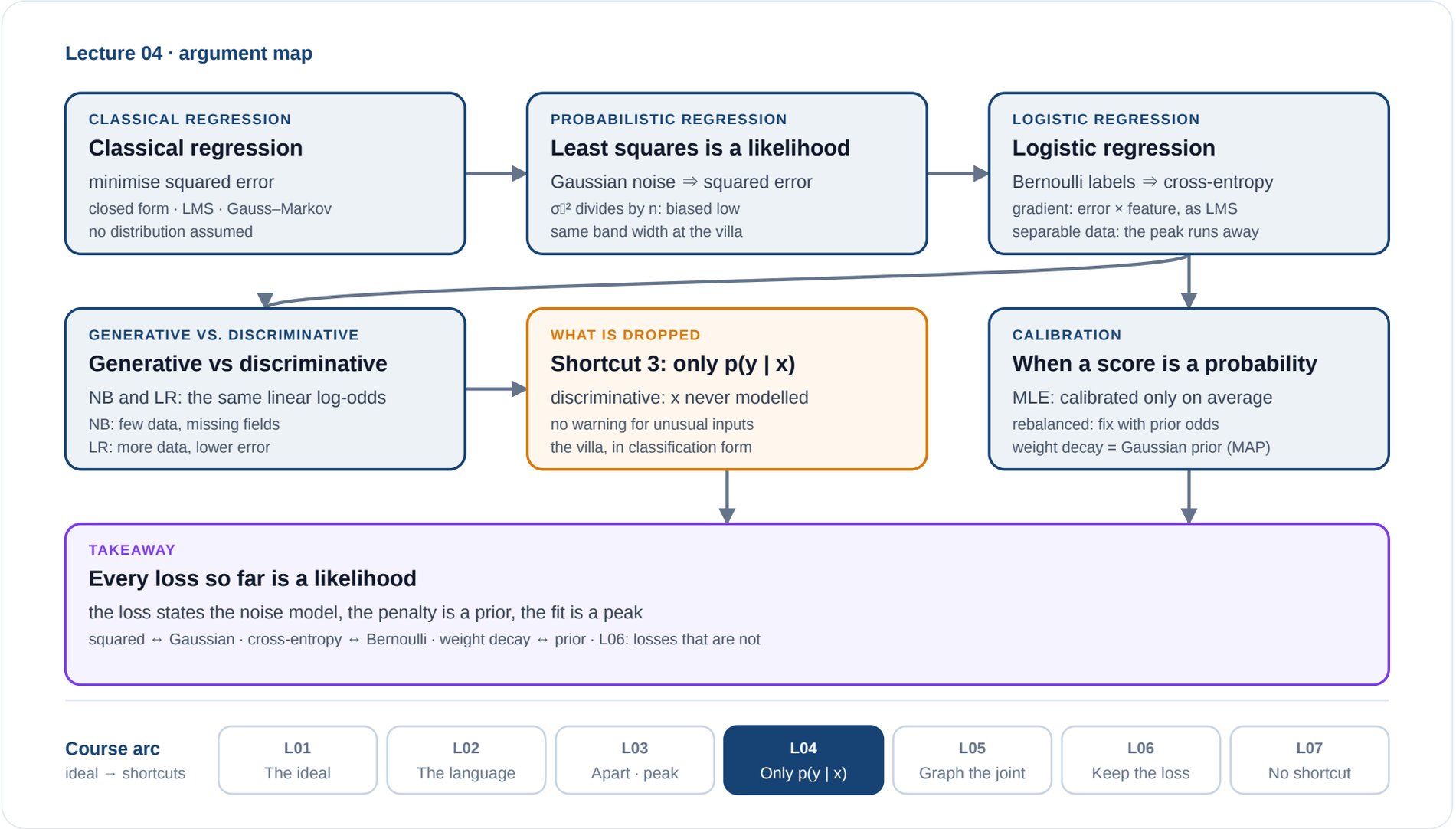
Today we meet the methods you know from earlier courses — linear and logistic regression — and read them as **probability models**. They never model the inputs:

$$p(y \mid x) \quad \text{instead of} \quad p(x, y) = p(y) \cdot p(x \mid y)$$

SHORTCUT 3

Only what you need: model the label given the inputs — *discriminative*.

Argument Map



Classical Linear Regression

Guiding Question: You have fitted lines by least squares many times. What exactly is the method, and what justifies it?

Classification versus Regression

Classification

Y is a finite set of labels.

Will this customer default — yes or no?

Regression

$Y = \mathbb{R}$, the target is a quantity.

What credit line should this customer receive?

The learning machinery is the same; only the shape of Y — and therefore the loss — changes.

The Model

Linear regression predicts the target as a weighted sum of the features:

$$h_{\mathbf{w}}(\mathbf{x}) = w_0 + w_1 x_1 + \ldots + w_p x_p = \mathbf{w}^\top \mathbf{x}$$

with $x_0 = 1$ absorbing the intercept.

The hypothesis space is $\mathbb{R}^{p+1}$: one hypothesis per weight vector.

The Objective: Residual Sum of Squares

$$\hat{\mathbf{w}} = \arg\min_{\mathbf{w}\in\mathbb{R}^{p+1}} \mathrm{RSS}(\mathbf{w}), \qquad \mathrm{RSS}(\mathbf{w}) = \sum_{i=1}^{n} \left(y_i - \mathbf{w}^\top \mathbf{x}_i\right)^2$$

RSS measures **effectiveness** — how good the predictions are. Do not confuse it with **efficiency**, which is about runtime and computational effort.

One Feature: The Closed Form

For a single feature the minimiser can be written down directly:

Slope

$$\hat{w}_1 = \frac{\sum_i (x_i - \overline{x})(y_i - \overline{y})}{\sum_i (x_i - \overline{x})^2}$$

The covariance of x and y over the variance of x.

Intercept

$$\hat{w}_0 = \overline{y} - \hat{w}_1 \overline{x}$$

The fitted line passes through the point of means.

How Much Variance Is Explained?

$$\mathrm{TSS} = \mathrm{ESS} + \mathrm{RSS}, \qquad R^2 = \frac{\mathrm{ESS}}{\mathrm{TSS}} = 1 - \frac{\mathrm{RSS}}{\mathrm{TSS}}$$

where TSS is the total, ESS the explained and RSS the residual sum of squares (the split holds for a least-squares fit with an intercept).

R^2 is the fraction of the target's variation the model accounts for: 0 means no better than predicting the mean, 1 means an exact fit.

Training R^2 never decreases when a feature is added, however useless the feature. A high training R^2 is not evidence that a model generalises — test R^2 and adjusted R^2 can and do fall.

Higher Dimensions

With p features the same objective in matrix form,

$$\hat{\mathbf{w}} = (\mathbf{X}^\top \mathbf{X})^{-1} \mathbf{X}^\top \mathbf{y}$$

gives the optimum in one step whenever $\mathbf{X}^\top \mathbf{X}$ is invertible.

> Invertibility fails when features are linearly dependent or when $p > n$ — more unknowns than equations. Both are common in practice; regularisation is the standard remedy.

Takeaway

Linear regression fits a weighted sum by minimising squared residuals. Its solution is built from **covariance over variance**, and R^2 reports how much variation it explains — not whether it will generalise.

Linear in the Parameters, Not in the Features

A model is **linear** when it is linear in the **weights** $\mathbf{w}$. The features may be arbitrary transformations of the input.

Plain

$$h(\mathbf{x}) = w_0 + w_1 x$$

a straight line

Polynomial

$$h(\mathbf{x}) = w_0 + w_1 x + w_2 x^2$$

a curve — still a *linear model*

Basis functions

$$h(\mathbf{x}) = \sum_j w_j \phi_j(\mathbf{x})$$

any fixed feature map ϕ

What *linear* forbids: a transformation that has to be learned from the data. The feature map is fixed before fitting; only the weights are learned.

Consequences

The good

The closed-form solution and all statistical guarantees carry over unchanged.

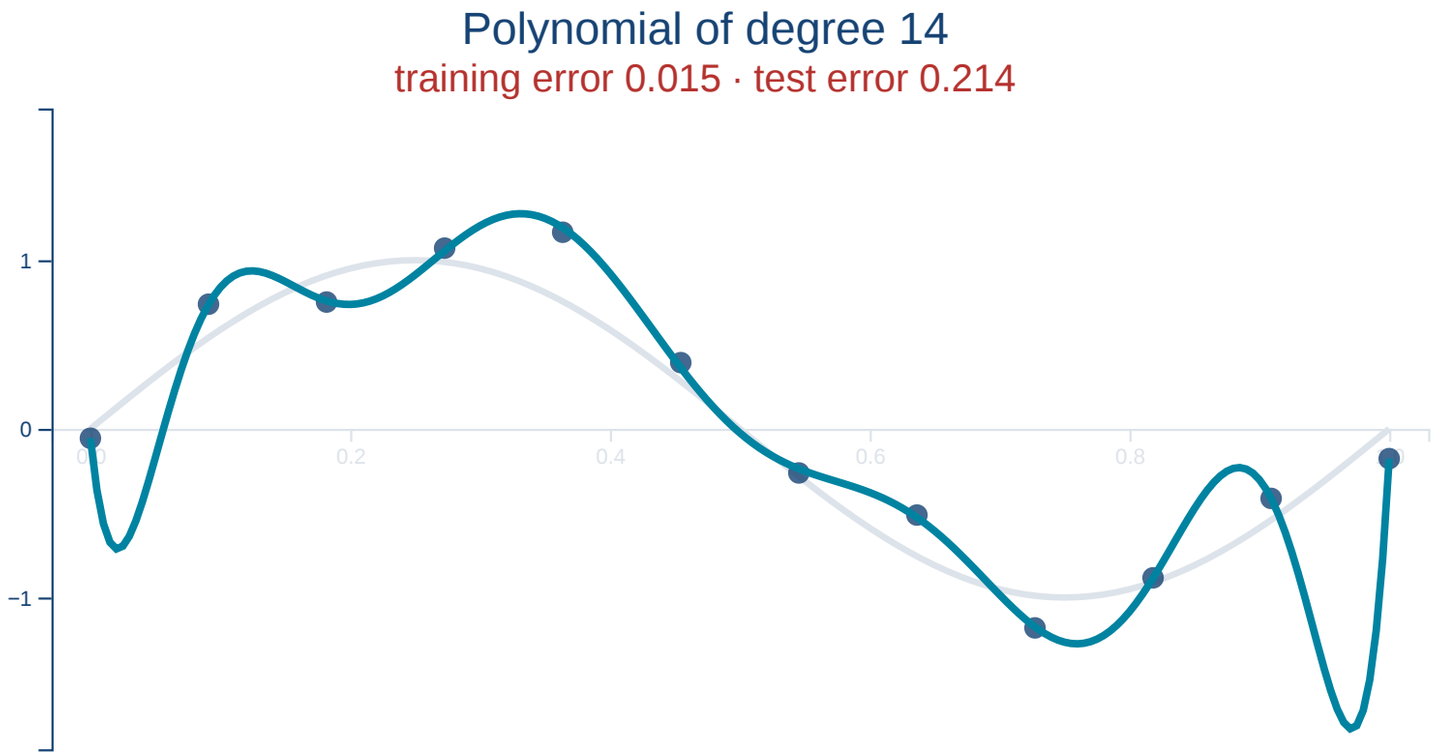
The catch

The feature map is a **modelling decision**. Choose too rich a basis and the model overfits; too poor and it underfits.

> Increasing the polynomial degree turns a dial on the bias–variance axis.

Turning the Capacity Dial

Linear in the parameters, any basis in the features: a polynomial of degree k is still "linear regression" — and k is a dial.



Fitting h by Repeated Correction

LMS (least mean squares)

1. Initialise the weights w.
2. Pick a training example $(\mathbf{x}, y)$ from the training set D.
3. Compute the prediction $h_w(\mathbf{x})$ and the error $e = y - h_w(\mathbf{x})$.
4. Update every weight: $w_i \leftarrow w_i + \eta e x_i$.
5. Repeat 2–4 until the error on D stops decreasing.

What the Update Rule Encodes

Direction

The sign of e decides whether the prediction goes up or down.

Attribution

The factor x_i gives each feature credit proportional to its value.

Step size

The learning rate η trades speed against stability.

Too large an η oscillates or diverges; too small an η converges slowly. It is a hyperparameter — chosen, not learned.

Two Ways to the Same Minimum

Direct (closed form)

$$\hat{\mathbf{w}} = (\mathbf{X}^\top \mathbf{X})^{-1} \mathbf{X}^\top \mathbf{y}$$

Exact, one step. Requires forming and inverting a $(p+1) \times (p+1)$ matrix — costly for large p, impossible when it is singular.

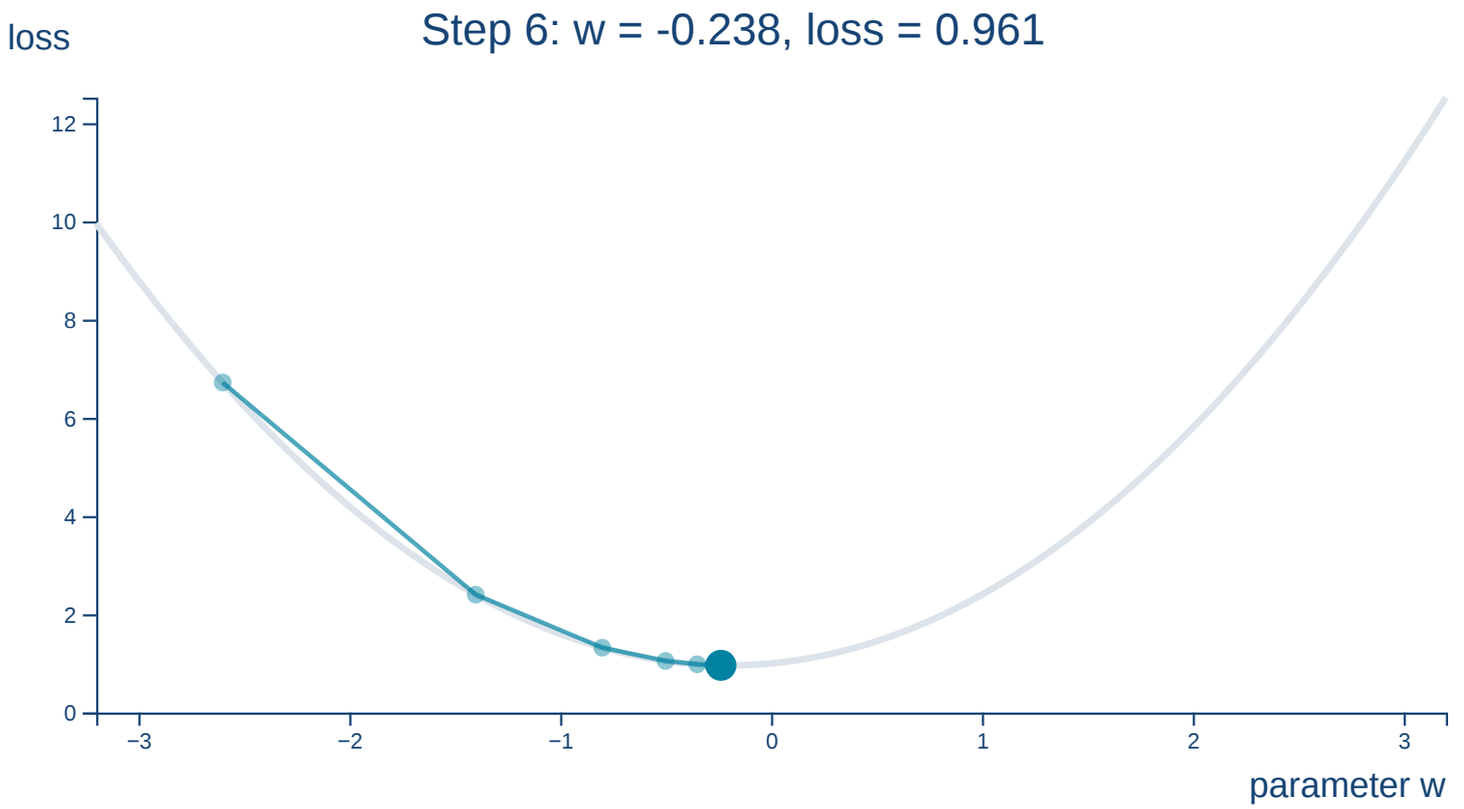
Iterative (gradient descent)

$$\mathbf{w} \leftarrow \mathbf{w} - \eta \nabla \mathrm{RSS}(\mathbf{w})$$

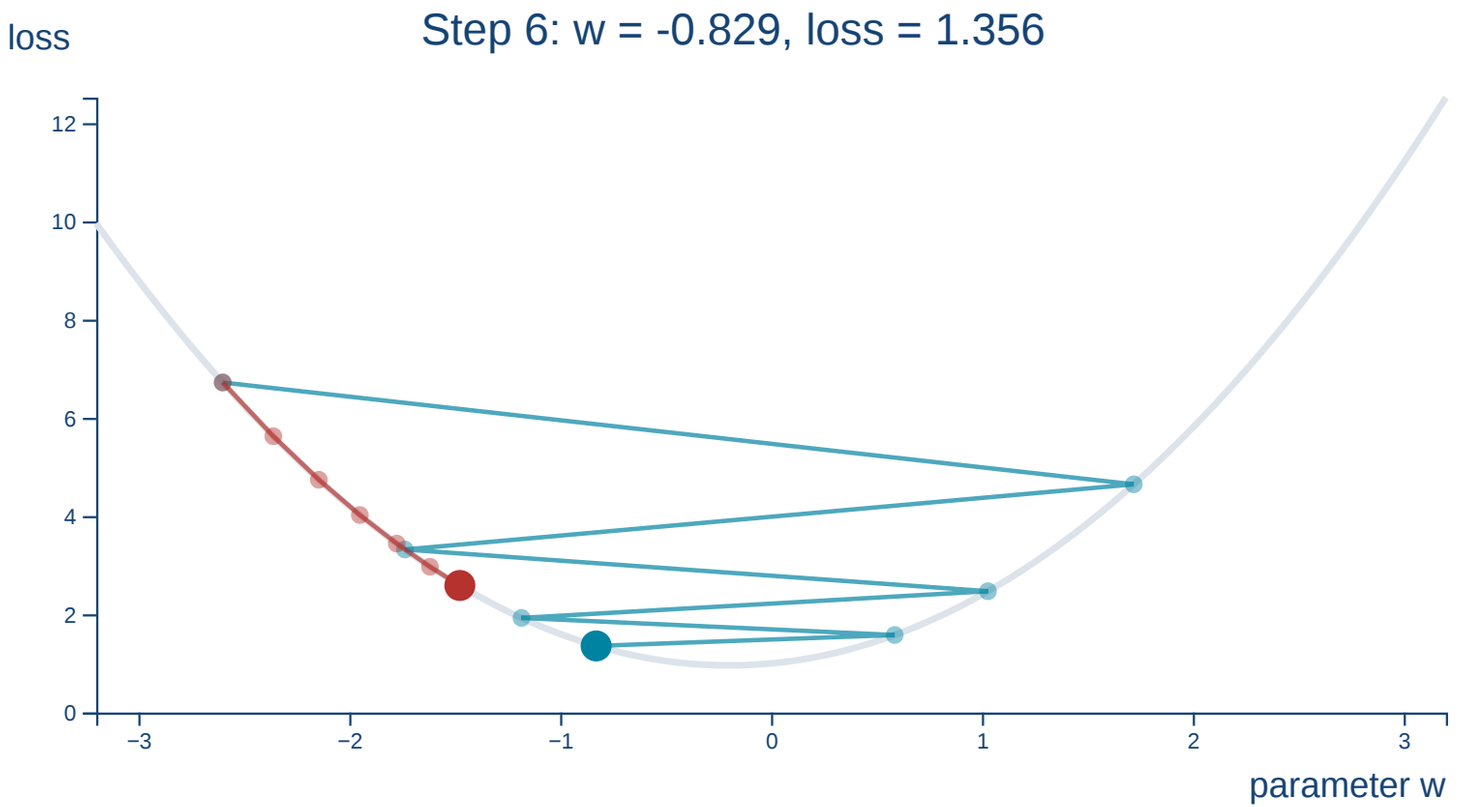
Approximate, many steps. Scales to large p and n, works when the direct solution does not exist, and extends to models with no closed form at all.

The iterative rule is exactly the **LMS algorithm**: its update $w_i \leftarrow w_i + \eta e x_i$ *is* a gradient step on RSS, taken per example instead of per dataset (the factor 2 is absorbed into η).

Watching Gradient Descent Work



The Learning Rate Decides Everything



Teal: $\eta = 0.9$ — overshoots and oscillates across the minimum. Red: $\eta = 0.05$ — stable, but slow: after six steps it has covered about half the distance.

The learning rate is a **hyperparameter**: it is chosen, not learned, and both failure modes are silent — one looks unstable, the other looks converged.

Takeaway

> Use the **closed form** when the matrix is small and well-conditioned; use **gradient descent** when it is not — and for most models beyond linear regression, no closed form exists at all.

The Question Nobody Asks

We minimised the *squared* residuals. Why squares — why not absolute deviations, or fourth powers?

Geometric View

The predictions $\mathbf{Xw}$ live in the **column space** of $\mathbf{X}$ — everything the features can express. The observed $\mathbf{y}$ generally does not.

Least squares picks the point of that space **closest to $\mathbf{y}$ in Euclidean distance**: the orthogonal projection. The residual is perpendicular to every feature.

Consequence

The residual carries no remaining linear information about the features.

The Gauss–Markov Theorem

Gauss–Markov (informal). Among all estimators that are

- **linear** in the observed outputs $(y_1, \ldots, y_n)$, and
- **unbiased**,

the least-squares estimator has the **lowest variance**. It is the *Best Linear Unbiased Estimator* — BLUE.

Best

lowest variance among the candidates

Linear

a linear function of the observations

Unbiased

right on average over repeated samples

What This Buys, and What It Costs

The payoff

Minimising RSS yields optimal statistical properties under **mild assumptions** — no normality needed, only a correctly specified linear model, linearly independent features, and zero-mean, uncorrelated errors of constant variance.

The price

Alternatives — absolute error, Huber loss — are **more robust to outliers** but give up the BLUE guarantee.

Squared error is not an arbitrary convenience: it is the loss whose minimiser is the best linear unbiased estimator. Departing from it is legitimate — but it is a trade, and you should be able to name what you traded.

Probabilistic Linear Regression

Guiding Question: You have minimised squared error many times. What did that choice assume about the world — and what does it leave out?

The Same Model, Told Differently

Classical account

Choose the line that **minimises squared error**:

$$\hat{\mathbf{w}} = \arg\min_{\mathbf{w}} \sum_i (y_i - \mathbf{w}^\top \mathbf{x}_i)^2$$

Probabilistic account

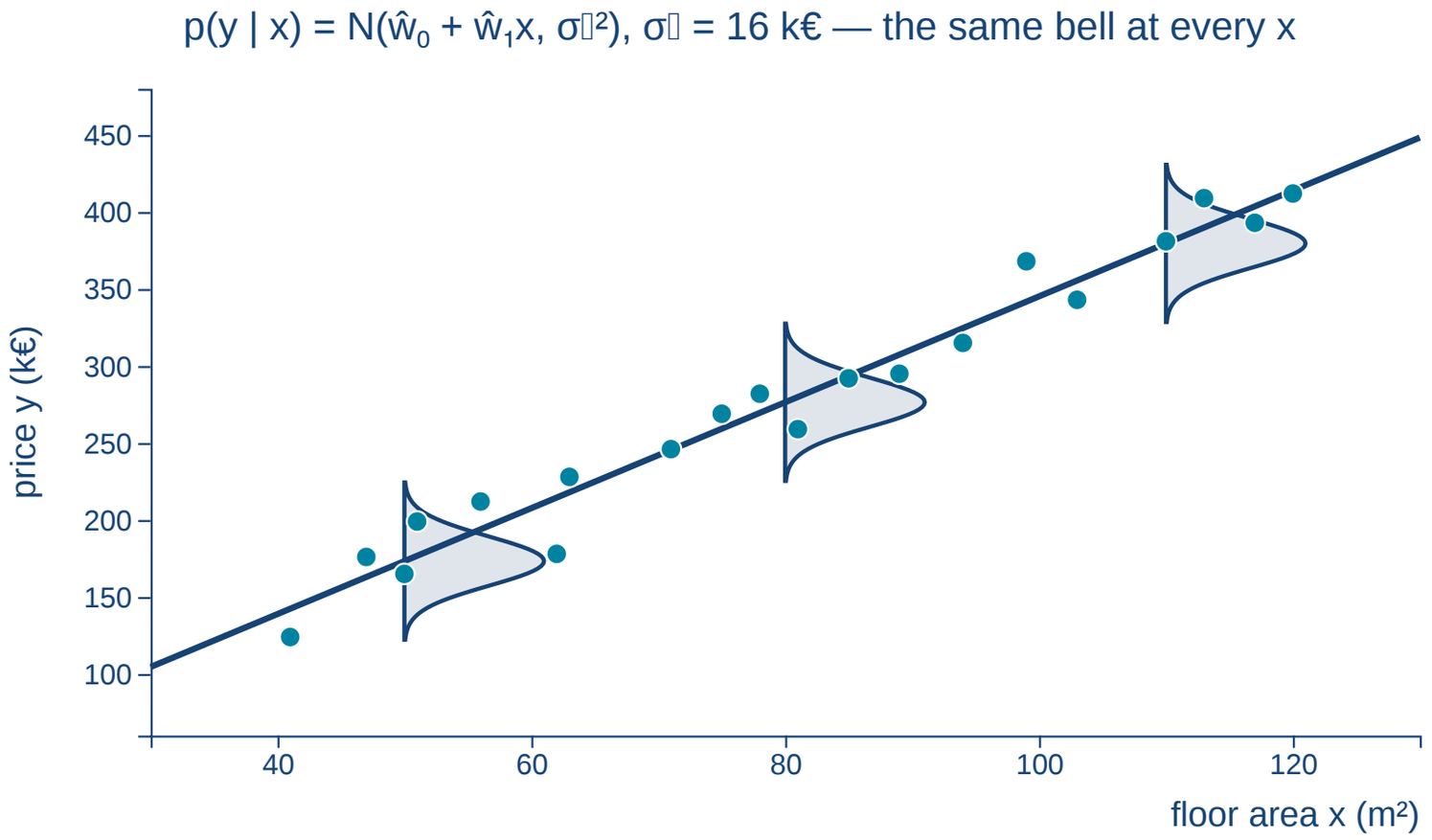
State how a price **arises** for a given flat:

$$y = \mathbf{w}^\top \mathbf{x} + \varepsilon, \qquad \varepsilon \sim \mathcal{N}(0, \sigma^2)$$

then keep the peak of the likelihood.

Do the two accounts give the same $\hat{\mathbf{w}}$ — and if so, what did the classical account assume without saying it?

A Distribution at Every x



A **conditional distribution** $p(y \mid x) = \mathcal{N}(\mathbf{w}^\top \mathbf{x}, \sigma^2)$: one bell per x, **equally wide**. Nothing about how x arises.

The Derivation

With $p(y_i \mid \mathbf{x}_i, \mathbf{w}) = \mathcal{N}(y_i \mid \mathbf{w}^\top \mathbf{x}_i, \sigma^2)$ and i.i.d. rows:

$$\ell(\mathbf{w}) = \sum_{i=1}^{n} \log \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(-\frac{(y_i - \mathbf{w}^\top \mathbf{x}_i)^2}{2\sigma^2}\right) = \underbrace{-\frac{n}{2}\log(2\pi\sigma^2)}_{\text{no } \mathbf{w}} - \frac{1}{2\sigma^2}\mathrm{RSS}(\mathbf{w})$$

> Maximising the Gaussian log-likelihood **is** minimising RSS: $\hat{\mathbf{w}}_{\mathrm{ML}} = \hat{\mathbf{w}}_{\mathrm{OLS}}$. Squared error is the exponent of the bell.

What Squared Error Assumed

the noise model says	squared error silently assumes	check on the residuals
ε has mean 0	the line is the right shape	no pattern against x
one σ for every x	**homoscedastic** noise	no fan shape
rows independent	i.i.d. (Lecture 02)	no pattern in time or space
ε Gaussian	thin tails: large errors are rare	no heavy tails, few outliers

Choosing a loss and choosing a noise model are **one decision**. Squared error ⇔ Gaussian noise; absolute error ⇔ Laplace noise — heavier tails, so a large residual is less surprising and an outlier pulls the fit less.

The Peak of σ^2

The same likelihood also has a peak in σ^2:

$$\frac{\partial \ell}{\partial \sigma^2} = -\frac{n}{2\sigma^2} + \frac{\mathrm{RSS}}{2\sigma^4} = 0 \quad \Longrightarrow \quad \hat{\sigma}^2_{\mathrm{ML}} = \frac{\mathrm{RSS}}{n}$$

Which of Lecture 02's two variances is this — and is it too small or too large?

The Peak of σ^2

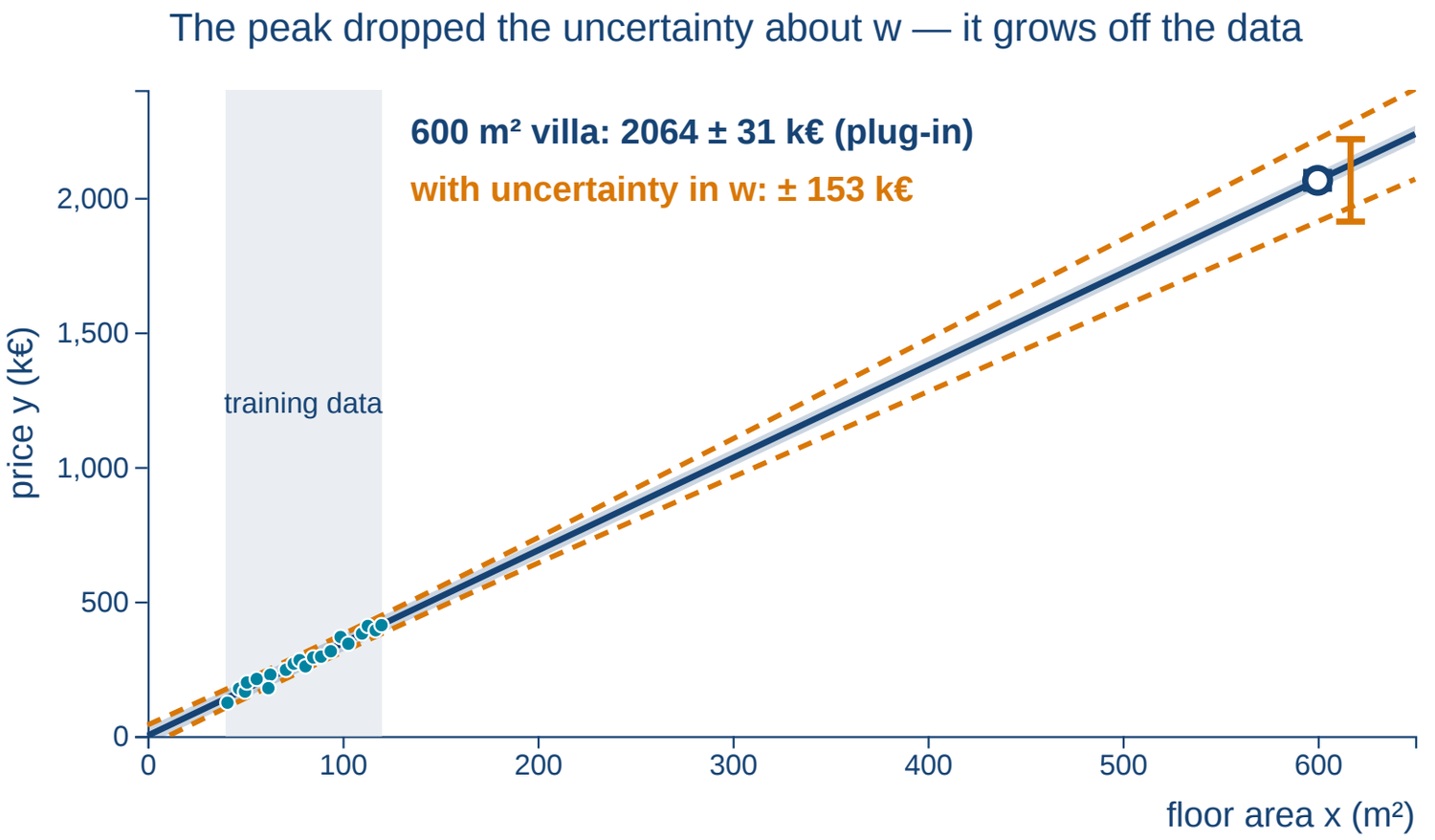
The same likelihood also has a peak in σ^2:

$$\frac{\partial \ell}{\partial \sigma^2} = -\frac{n}{2\sigma^2} + \frac{\mathrm{RSS}}{2\sigma^4} = 0 \quad \Longrightarrow \quad \hat{\sigma}^2_{\mathrm{ML}} = \frac{\mathrm{RSS}}{n}$$

Which of Lecture 02's two variances is this — and is it too small or too large?

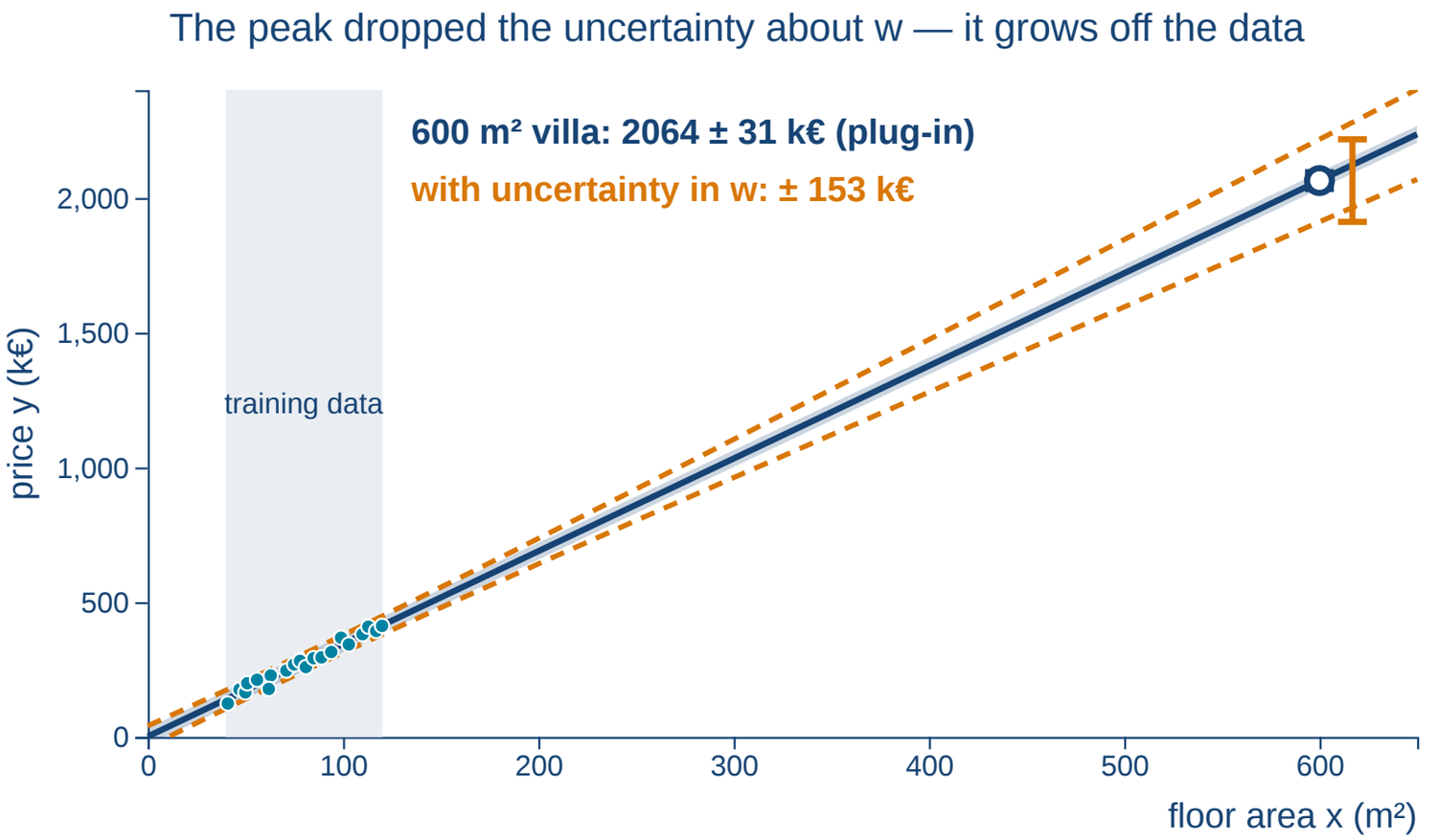
> The $1/n$ one. The line was fitted to the same data, so residuals are smaller than the true errors: $\hat{\sigma}^2_{\mathrm{ML}}$ is **biased low**. The unbiased version divides by $n - p$ for p fitted weights. Houses: $n = 20$, $p = 2$, RSS $\approx 5{,}097 \rightarrow \hat{\sigma} \approx 16.0$ instead of 16.8 k€.

The Villa Again



The band is as narrow at the villa as at a flat. Is the model as sure?

The Villa Again



The band is as narrow at the villa as at a flat. Is the model as sure?

No: ± 31 k€ is only the noise $\hat{\sigma}$. The uncertainty about $\mathbf{w}$ — dropped with the peak — grows off the data: ± 153 k€ at the villa (Lecture 07).

Mini-Exercise

The residuals of your house-price model fan out: about ± 10 k€ for 40 m² flats, ± 60 k€ for 120 m² flats.

Which assumption fails — and which output of the model can you no longer trust?

Mini-Exercise

The residuals of your house-price model fan out: about ± 10 k€ for 40 m² flats, ± 60 k€ for 120 m² flats.

Which assumption fails — and which output of the model can you no longer trust?

One σ for every x (homoscedasticity). The line may still be fine, but the **band** is not: a single $\hat{\sigma}$ is too wide for small flats and too narrow for large ones.

Takeaway

Least squares **is** maximum likelihood under Gaussian noise: the loss states the noise model, and the model is a distribution $p(y \mid x)$. It is also a **peak**: $\hat{\sigma}^2$ divides by n, and the band is as narrow at the villa as at a flat — the uncertainty about $\mathbf{w}$ was dropped.

Logistic Regression

Guiding Question: Lecture 01's fraud model said 0.55. Where does such a number come from — and what does the model refuse to know?

Why Not Regress on 0/1?

Code fraud as $y = 1$, legitimate as $y = 0$, and fit the line of probabilistic linear regression.

Why Not Regress on 0/1?

Code fraud as $y = 1$, legitimate as $y = 0$, and fit the line of probabilistic linear regression.

Squared error punishes being right

A transaction far on the correct side has a large residual, so least squares drags the boundary towards cases it already classifies correctly.

The output is not a probability

$\mathbf{w}^\top \mathbf{x}$ can be 17 or -40. The Gaussian bell of the regression model puts mass on every real y — for a label that is 0 or 1.

Keep the linear score, change what it models: the **log-odds**, and a noise model that produces 0s and 1s.

Log-Odds Are Linear

$$\ln \frac{P(y=1 \mid \mathbf{x})}{1-P(y=1 \mid \mathbf{x})} = \mathbf{w}^\top \mathbf{x} \qquad \Longleftrightarrow \qquad P(y=1 \mid \mathbf{x}) = \sigma(\mathbf{w}^\top \mathbf{x}) = \frac{1}{1+e^{-\mathbf{w}^\top \mathbf{x}}}$$

read the weights as	fraud example
the intercept: the log-odds when every feature is 0	no features at all: $w_0 = \ln \frac{1}{999} \approx -6.9$, the base rate 1 in 1,000
a weight: what one unit of a feature **adds** to the log-odds	"new device" with $w = \ln 6 \approx 1.8$: odds $\times 6$
the boundary: where $\mathbf{w}^\top \mathbf{x} = 0$	score 0.5 — still a straight line

Where Does Cross-Entropy Come From?

$$L(\mathbf{w}) = -\sum_{i=1}^{n} \Big[y_i \ln h_i + (1 - y_i) \ln(1 - h_i) \Big], \qquad h_i = \sigma(\mathbf{w}^\top \mathbf{x}_i)$$

You have minimised this loss many times. Which noise model does it state?

Where Does Cross-Entropy Come From?

$$L(\mathbf{w}) = -\sum_{i=1}^{n} \Big[y_i \ln h_i + (1 - y_i) \ln(1 - h_i) \Big], \qquad h_i = \sigma(\mathbf{w}^\top \mathbf{x}_i)$$

You have minimised this loss many times. Which noise model does it state?

Given $\mathbf{x}$, the label is a **Bernoulli** draw with bias h: $P(y_i \mid \mathbf{x}_i, \mathbf{w}) = h_i^{y_i}(1 - h_i)^{1 - y_i}$. With i.i.d. rows,

$$\ell(\mathbf{w}) = \sum_{i=1}^{n} \Big[y_i \ln h_i + (1 - y_i) \ln(1 - h_i) \Big] = -L(\mathbf{w})$$

> Cross-entropy **is** the Bernoulli negative log-likelihood — Lecture 03's coin, biased by $\mathbf{x}$. How $\mathbf{x}$ arises is never said.

The Same Update Again

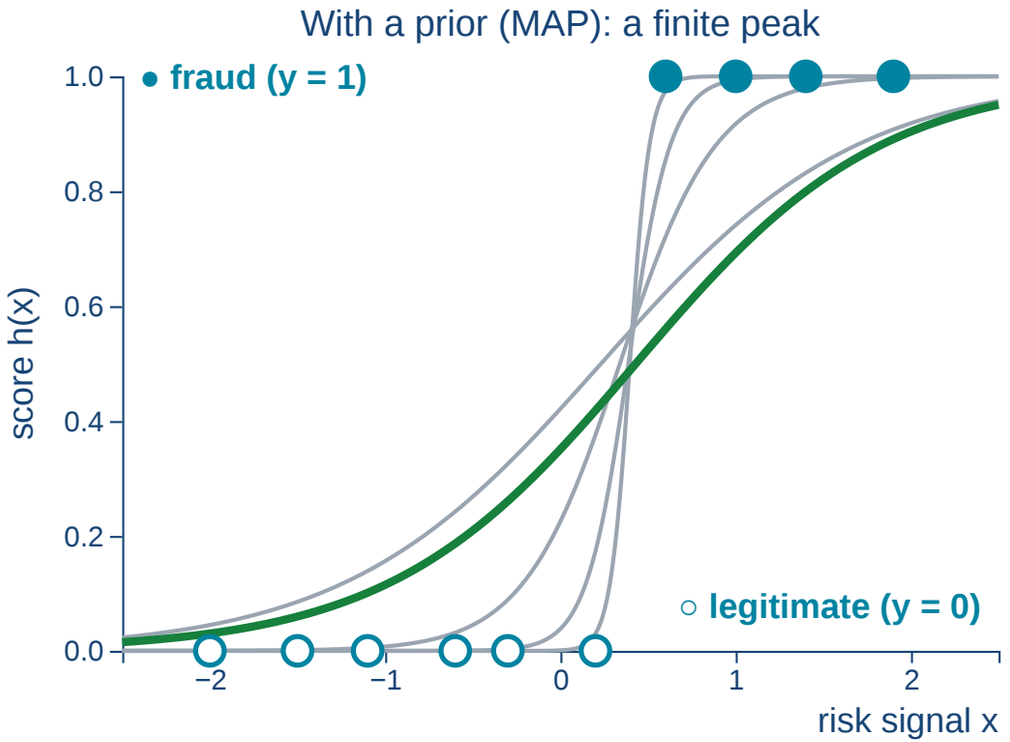
With $\sigma' = \sigma(1-\sigma)$, the gradient of the log-likelihood is

$$\frac{\partial \ell}{\partial \mathbf{w}} = \sum_{i=1}^{n} (y_i - h_i)\mathbf{x}_i$$

model	noise	loss	gradient of ℓ
linear regression	Gaussian	squared error	$\frac{1}{\sigma^2}\sum_i (y_i - \mathbf{w}^\top \mathbf{x}_i)\mathbf{x}_i$
logistic regression	Bernoulli	cross-entropy	$\sum_i (y_i - \sigma(\mathbf{w}^\top \mathbf{x}_i))\mathbf{x}_i$

Error × feature — the LMS update with a sigmoid in the prediction; no closed form, so iterate. A fraud scored 0.01 costs $-\ln 0.01 \approx 4.6$, scored 0.99 only 0.01: confident mistakes dominate.

When the Peak Runs Away

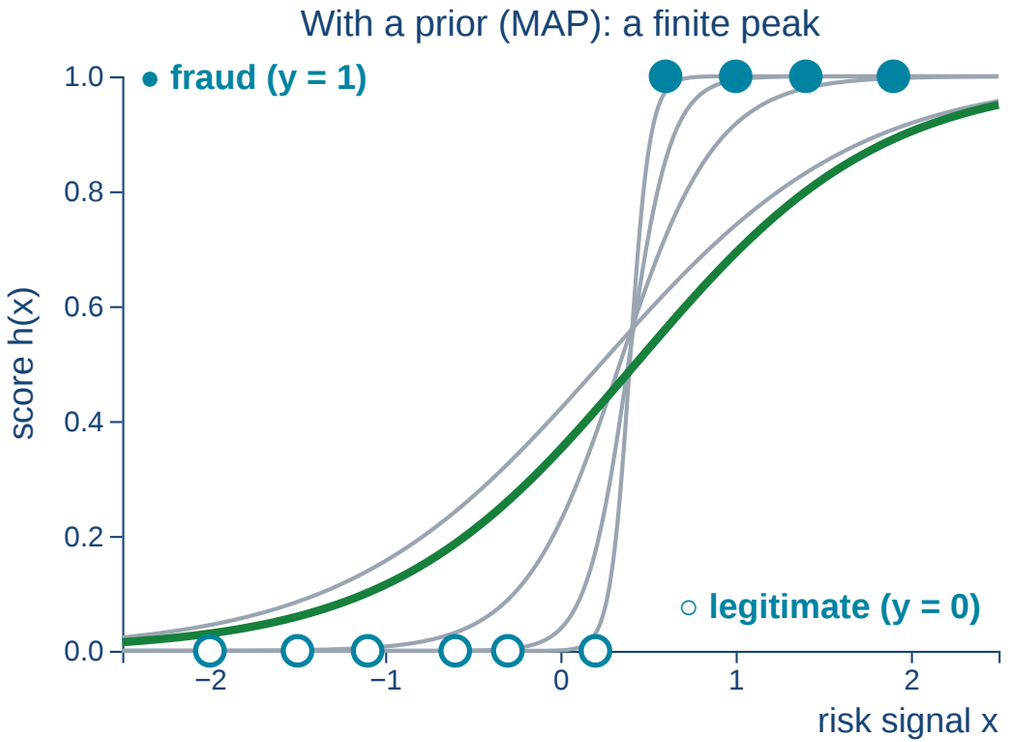


steps	slope	log-lik
10	1.4	-2.68
100	3.6	-1.04
1,000	8.3	-0.36
10,000	17.8	-0.06
MAP	**1.4**	**-2.54**

prior on the slope: N(0, 1)
stays, however long it runs

Every fraud scores above every legitimate case. Where is the peak of the likelihood?

When the Peak Runs Away



steps	slope	log-lik
10	1.4	-2.68
100	3.6	-1.04
1,000	8.3	-0.36
10,000	17.8	-0.06
MAP	**1.4**	**-2.54**

prior on the slope: N(0, 1)
stays, however long it runs

Every fraud scores above every legitimate case. Where is the peak of the likelihood?

Nowhere. The likelihood keeps rising as the slope grows; the scores harden to 0 and 1 — three heads in three flips. A prior's peak (MAP) is finite.

Shortcut 3: Model Only $p(y \mid \mathbf{x})$

Linear and logistic regression are **discriminative**: a model of y given $\mathbf{x}$, none of $\mathbf{x}$.

What is dropped: $p(\mathbf{x})$

- a transaction unlike any seen still gets a confident score
- a missing field cannot be averaged out — it must be filled in
- no way to simulate a plausible transaction

What is gained

- no assumptions about how features are distributed — where, with hundreds of features, most modelling errors hide
- every parameter serves the decision

Lecture 03 took the other route: model the joint $p(\mathbf{x}, y)$ and condition — **generative**. Generative vs. Discriminative Models compares the two.

Mini-Exercise

Your fraud model scores **0.97** for a transfer 50 times larger than any in its training data.

Should you trust the 0.97 — and could the model itself warn you?

Mini-Exercise

Your fraud model scores **0.97** for a transfer 50 times larger than any in its training data.

Should you trust the 0.97 — and could the model itself warn you?

Not blindly: the linear log-odds simply **extrapolate**, like the line at the villa. And no — the model has **no** $p(\mathbf{x})$, so nothing in it can see that the amount is unusual. That needs a model of $\mathbf{x}$ or a separate check.

Takeaway

Logistic regression is probabilistic linear regression with Bernoulli noise: linear log-odds, cross-entropy as the negative log-likelihood, error × feature as the gradient. It **keeps the peak** — which runs away on separable data — and **models only** $p(y \mid \mathbf{x})$: it never learns what a normal transaction looks like.

Generative vs. Discriminative Models

Guiding Question: Naive Bayes and logistic regression see the same fraud table. Which would you trust with 50 labelled frauds — and which with 50,000?

The Same Line, Fitted Twice

Take the logarithm of the naive-Bayes odds. For yes/no flags $x_j \in \{0, 1\}$ with $\theta_{jy} = P(x_j = 1 \mid y)$:

$$\log \frac{P(\text{fraud} \mid x)}{P(\text{honest} \mid x)} = w_0 + \sum_{j=1}^{p} w_j x_j, \qquad w_j = \log \frac{\theta_{j1}}{\theta_{j0}} - \log \frac{1 - \theta_{j1}}{1 - \theta_{j0}}$$

Where have you seen a log-odds that is linear in x?

The Same Line, Fitted Twice

Take the logarithm of the naive-Bayes odds. For yes/no flags $x_j \in \{0, 1\}$ with $\theta_{jy} = P(x_j = 1 \mid y)$:

$$\log \frac{P(\text{fraud} \mid x)}{P(\text{honest} \mid x)} = w_0 + \sum_{j=1}^{p} w_j x_j, \qquad w_j = \log \frac{\theta_{j1}}{\theta_{j0}} - \log \frac{1 - \theta_{j1}}{1 - \theta_{j0}}$$

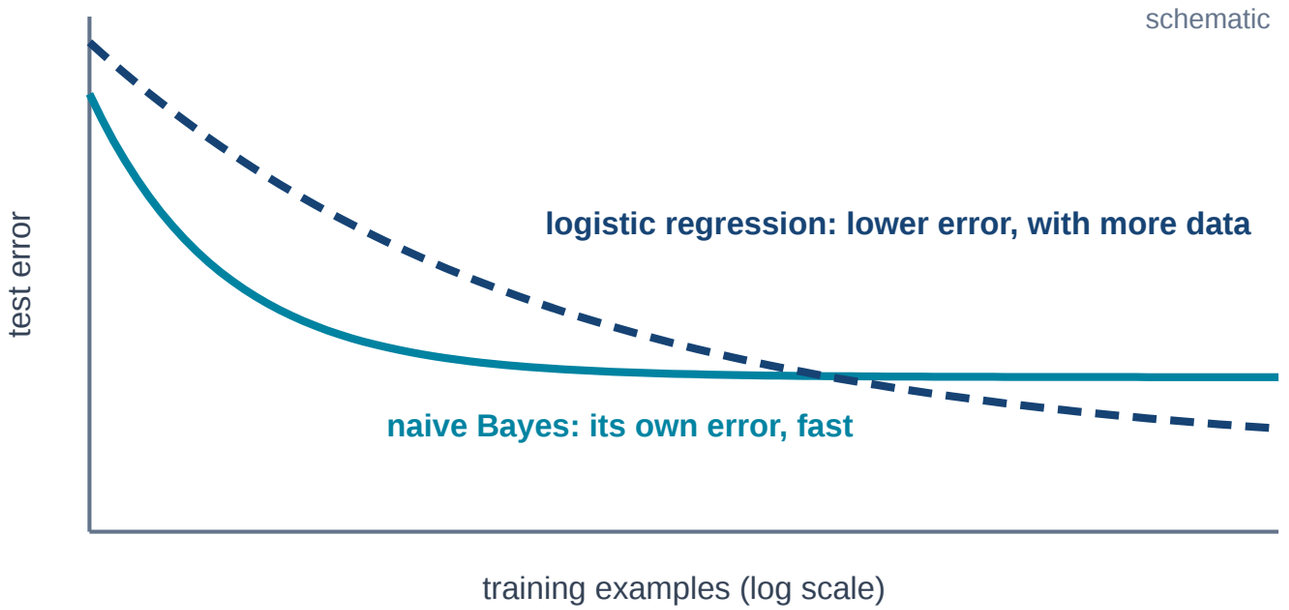
Where have you seen a log-odds that is linear in x?

> **Logistic regression** (previous module): $P(\text{fraud} \mid x) = \sigma(w_0 + w^\top x)$. Same family of classifiers — naive Bayes sets the weights by **counting**, one attribute at a time; logistic regression by **maximising** $\sum_i \log p(y_i \mid x_i)$, all weights jointly.

Two Ways to a Classifier

	naive Bayes — generative	logistic regression — discriminative
models	$P(y)$ and $p(x \mid y)$	$P(y \mid x)$ only
fitted by	counting (joint likelihood)	iterative optimisation (conditional likelihood)
correlated flags	counted twice	credit shared
a missing field	drop its factor	needs a value
an unusual x	low $p(x)$: can be flagged	still a confident score
a new base rate	change $P(y)$	refit or correct the intercept
drops	dependences between attributes	everything about x

Few Data, Many Data



Few labelled frauds: naive Bayes. Its assumption acts like a strong prior — little to estimate, so it settles fast.
Many labelled frauds: logistic regression. It pays for fewer assumptions with data, and ends lower.

Shown empirically and in theory by Ng & Jordan (2002). A wrong assumption costs accuracy in the limit and buys it early — the prior of Lecture 03, again.

Mini-Exercise

Our three-flag transaction arrives **without the IP field** — the logging failed.

What does naive Bayes do with it — and what does logistic regression do?

Mini-Exercise

Our three-flag transaction arrives **without the IP field** — the logging failed.

What does naive Bayes do with it — and what does logistic regression do?

Naive Bayes drops the factor: $\sum_v p(x_{\text{IP}} = v \mid y) = 1$, so the odds become $6 \cdot 2.5 : 999$ — about **1.5%**. **Logistic regression** has no model of x: it needs *some* value for x_{IP}, so someone must impute one, or train a second model without the field.

Takeaway

> Naive Bayes and logistic regression draw **the same kind of line**; they differ in what they model. The generative model pays with assumptions about x and gets answers about x; the discriminative one pays with data.

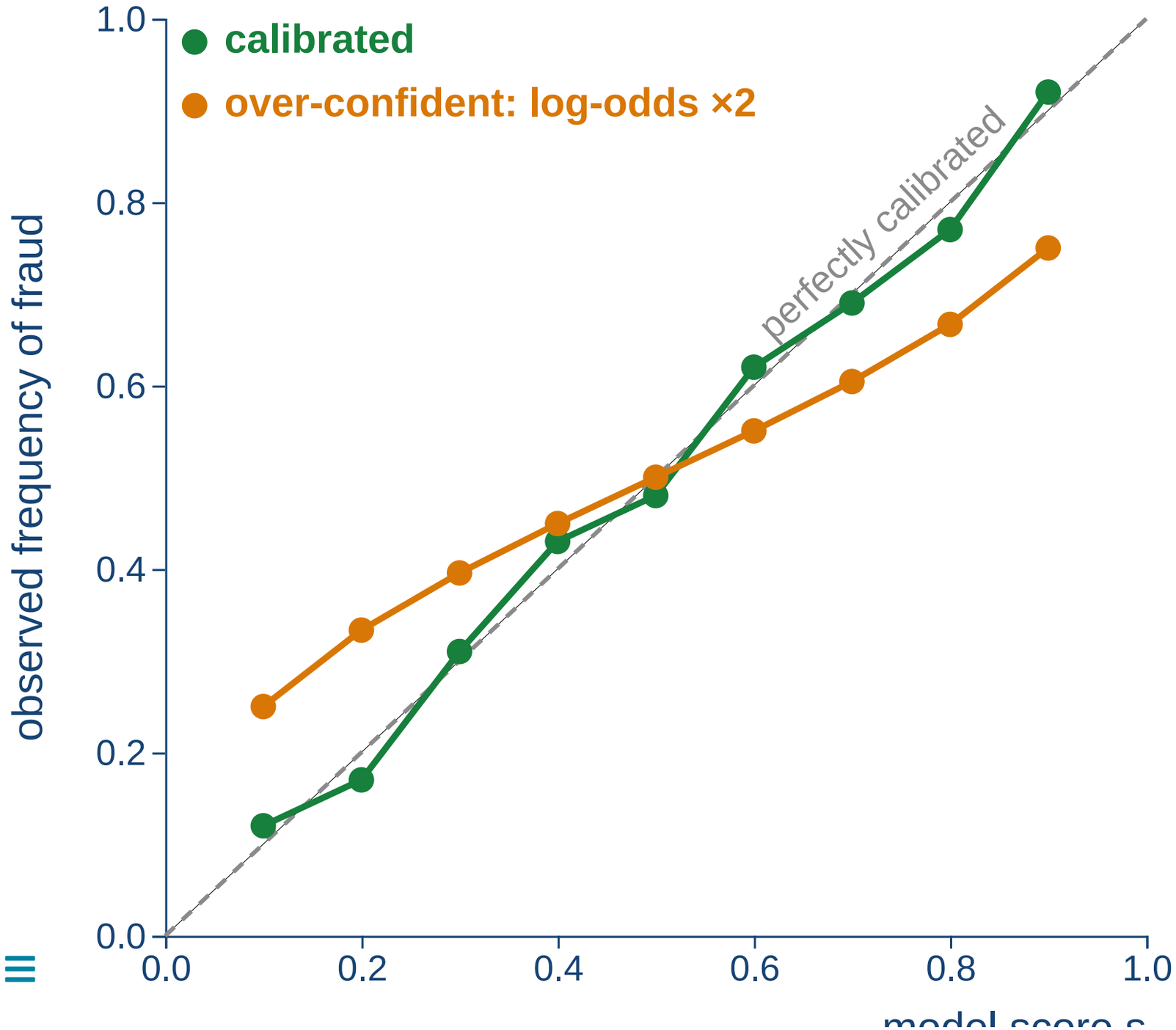
lecture	shortcut	what is dropped
Lecture 03	keep the peak	how sure we are
Lecture 03	assume it apart	every dependence between attributes
Lecture 04	only $p(y \mid x)$	every question about x

Naive Bayes is a graph with one hub. **Lecture 05** draws the others — and keeps exactly the dependences we claim.

Calibration

Guiding Question: Lecture 01 promised to say when a model's output earns the name "probability". When does it — and what breaks it?

When Is 0.7 a Probability?



A score is **calibrated** if, among all cases scored s, a fraction s is positive. A **reliability diagram** plots that fraction against s.

Predict before click 2: a model whose log-odds are twice too large — above or below the diagonal at high scores?

What Maximum Likelihood Guarantees

At the peak, the gradient for the intercept ($x_0 = 1$) is zero:

$$\sum_{i=1}^{n}(y_i - h_i) = 0 \quad \Longrightarrow \quad \frac{1}{n}\sum_{i=1}^{n} h_i = \frac{1}{n}\sum_{i=1}^{n} y_i$$

What does this equation promise — and what does it not?

What Maximum Likelihood Guarantees

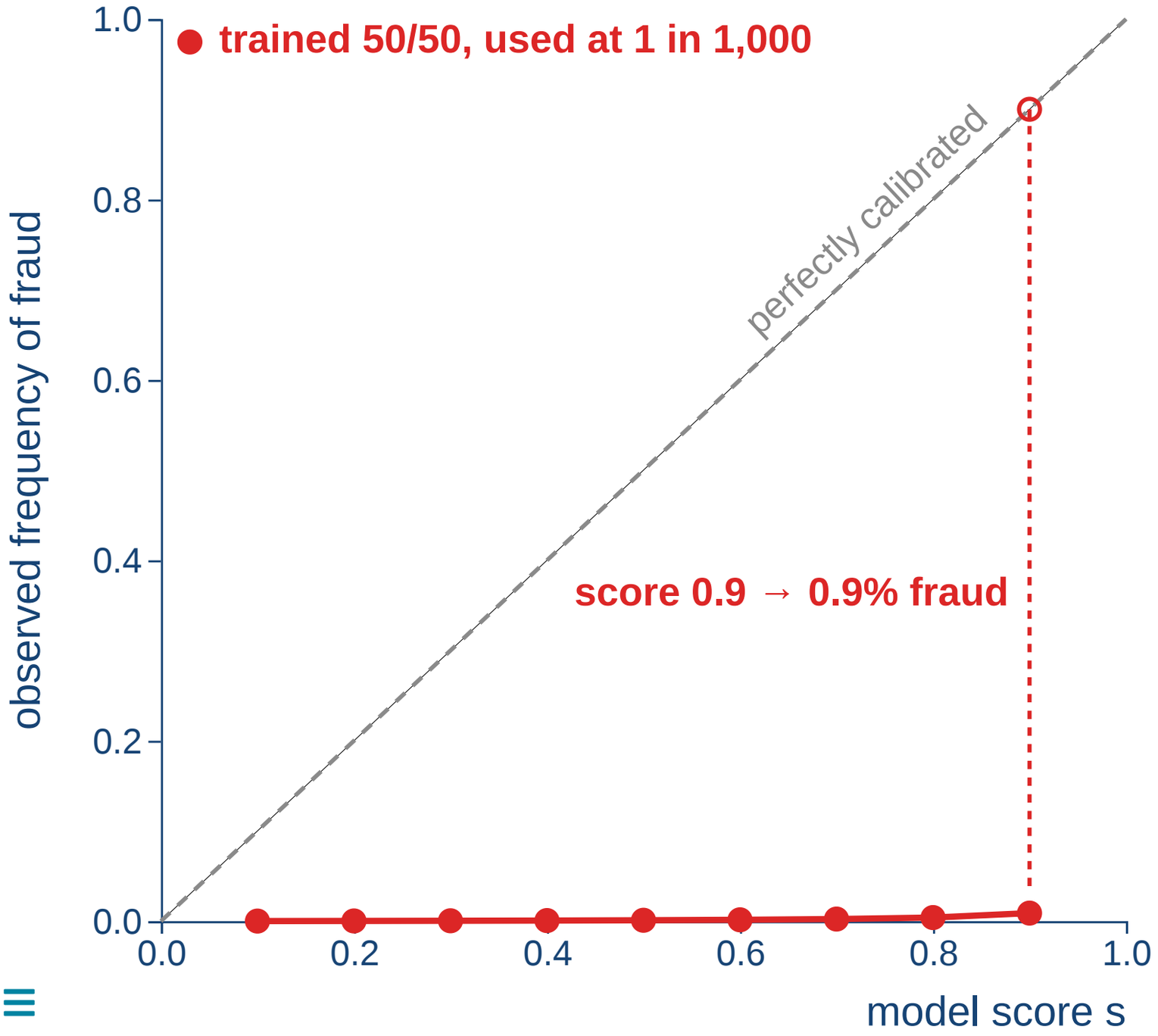
At the peak, the gradient for the intercept ($x_0 = 1$) is zero:

$$\sum_{i=1}^{n}(y_i - h_i) = 0 \quad \Longrightarrow \quad \frac{1}{n}\sum_{i=1}^{n} h_i = \frac{1}{n}\sum_{i=1}^{n} y_i$$

What does this equation promise — and what does it not?

promised	not promised
average score = training fraud rate: *calibration in the large*	each score right — needs log-odds truly linear in $\mathbf{x}$
on the **training** distribution	after a base-rate shift, or **rebalancing**
	that it **separates**: the base rate for everyone is calibrated — and useless

Rebalanced Training



Fraud is 1 in 1,000, so practitioners **undersample** legitimate cases to 50/50. The model learns odds for a world where fraud is common.

> Bayes in odds form:
>
> $$\text{real odds} = \text{model odds} \times \frac{1/999}{1}$$
>
> Score 0.9: odds $9 \to 9/999$, about **0.9%**.

Regularisation Is a Prior

$$\min_{\mathbf{w}} \mathrm{RSS}(\mathbf{w}) + \lambda \|\mathbf{w}\|^2$$

You know this as ridge regression or weight decay. Which prior's peak is it?



Regularisation Is a Prior

$$\min_{\mathbf{w}} \mathrm{RSS}(\mathbf{w}) + \lambda \|\mathbf{w}\|^2$$

You know this as ridge regression or weight decay. Which prior's peak is it?

MAP from Lecture 03 with $\mathbf{w} \sim \mathcal{N}(0, \tau^2 \mathbf{I})$: $\log p(\mathbf{w} \mid D) = -\frac{\mathrm{RSS}}{2\sigma^2} - \frac{\|\mathbf{w}\|^2}{2\tau^2} + \mathrm{const}$, so $\lambda = \sigma^2/\tau^2$.

penalty	prior on $\mathbf{w}$	what its peak does
$\lambda \|\mathbf{w}\|^2$	Gaussian	shrinks; keeps separable data finite
$\lambda \|\mathbf{w}\|_1$	Laplace	exact zeros — a property of the **peak** (the kink at 0); the posterior mean has none

Mini-Exercise

A fraud model trained on a **50/50** sample is used on real traffic, where fraud is **1 in 1,000**. It outputs **0.5** and **0.8**.

What are the real fraud probabilities?

Mini-Exercise

A fraud model trained on a **50/50** sample is used on real traffic, where fraud is **1 in 1,000**. It outputs **0.5** and **0.8**.

What are the real fraud probabilities?

Multiply the odds by $1/999$. Score 0.5: odds $1 \to 1/999$, about **0.1% —** the base rate itself. Score 0.8: odds $4 \to 4/999$, about **0.4%**.

Takeaway

A score is a probability when it is **calibrated** — checkable on a reliability diagram, not by accuracy. Maximum likelihood guarantees it only on average and only on the training distribution; **rebalancing** breaks it, and the prior odds repair it. **Regularisation is a prior**, and a penalised fit is a MAP estimate.

Every Loss So Far Is a Likelihood

So far — Lecture 06 meets losses that are not.

you minimised	it is $-\log$ of	the fit is
squared error	Gaussian noise around $\mathbf{w}^\top \mathbf{x}$	the peak (MLE)
cross-entropy	Bernoulli labels with bias $\sigma(\mathbf{w}^\top \mathbf{x})$	the peak (MLE)
$+\lambda \lVert \mathbf{w} \rVert^2$	a Gaussian prior on $\mathbf{w}$	a prior's peak (MAP)
$+\lambda \lVert \mathbf{w} \rVert_1$	a Laplace prior on $\mathbf{w}$	a prior's peak (MAP)

Dropped: the spread

how sure we are about $\mathbf{w}$ — **Lecture 07** keeps it

Dropped: $p(\mathbf{x})$

what a normal input looks like — **Lecture 03** modelled it, naive Bayes